

Fiduccia–Mattheyses bipartition

Fiduccia–Mattheyses moves one vertex at a time instead of swapping a pair

The idea

- Bucketed single-vertex gains, lock after each move
- Two legal flips can equal one KL swap's total gain on this instance

Pseudocode

- FM moves one vertex at a time
- Pseudocode ranks unlocked legal flips by gain
- Open this module's examples file and find the Pseudocode section
- That written sketch is what you implement on the implement track and what the browser

Algorithm sketch

- Same bad seed as KL

Algorithm sketch — try these

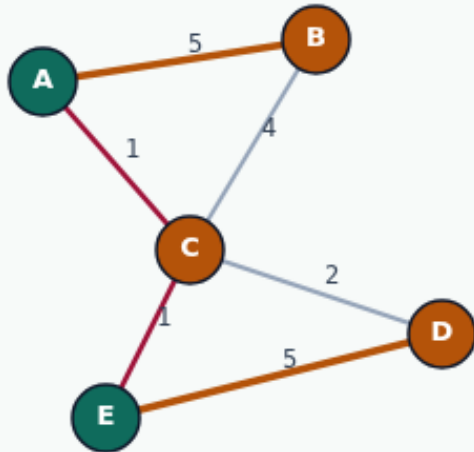
```
INPUT: side[], balance_tol, max_passes
OUTPUT: refined side[]
each pass: while unlocked legal moves exist:
    pick v with max gain among balance-ok flips
    lock v; flip on working copy
keep best positive-gain prefix; apply
stop when a pass cannot improve
GOLDEN BAD_SEED → flip D then A → cut 3
```

Same bad seed, different move set

FIDUCCIA-MATTHEYSES BIPARTITION

Same bad seed, different move set

Step 1 / 5 · bad-seed · module02-03-fm-partition



Explanation

FM starts from the same cutsize-12 seed, but moves one vertex at a time instead of swapping a pair. That suits hypergraph / cell-move implementations.

- Single-vertex moves across the cut
- Bucketed gains for speed (classic FM)
- Balance tolerance limits lopsided moves

seed cutsize: 12
parts: AE|BCD

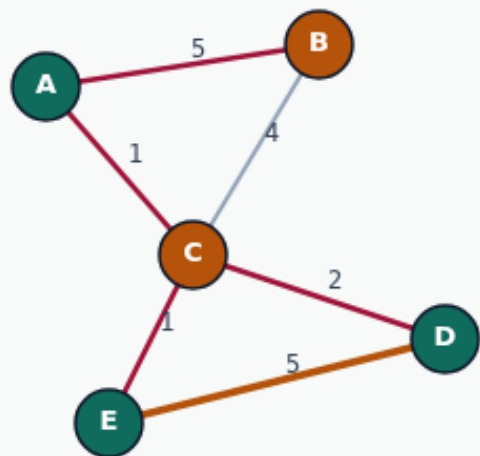
Same bad seed, different move set

Move 1: flip D (gain 3)

FIDUCCIA-MATTHEYSES BIPARTITION

Move 1: flip D (gain 3)

Step 2 / 5 · move-d · module02-03-fm-partition



Explanation

Highest legal move sends D to the other side with gain 3.
Partial progress: D joins E's side early.

- Pick unlocked vertex with best gain
- Apply move, lock vertex
- Update neighbor gains

move: D(g=3)
running toward best prefix

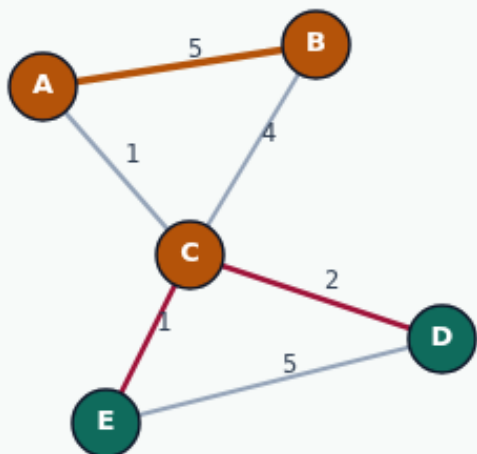
Move 1: flip D (gain 3)

Move 2: flip A (gain 6)

FIDUCCIA-MATTHEYSES BIPARTITION

Move 2: flip A (gain 6)

Step 3 / 5 · move-a · module02-03-fm-partition



Explanation

Next, A flips with gain 6. Cumulative gain is $3+6=9$ — the same total improvement KL found with one swap.

- Two moves $\approx$ one KL swap's worth of gain
- `best_k=2` keeps both moves
- `bestCum=9`

```
moves: D(3), A(6)
bestCum=9
cut 12-3
```

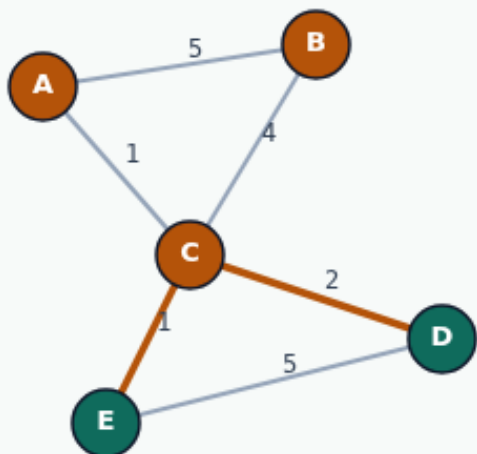
Move 2: flip A (gain 6)

Final ABC|DE, cutsize 3

FIDUCCIA-MATTHEYSES BIPARTITION

Final ABC|DE, cutsize 3

Step 4 / 5 · final · module02-03-fm-partition



Explanation

FM lands on the same refined bipartition as KL. Teaching point: move style differs, destination quality matches on this instance.

- parts: ABC|DE
- cutsize: 3
- Compare with KL transcript side-by-side

```
cutsize: 3  
pass 0 improved=true
```

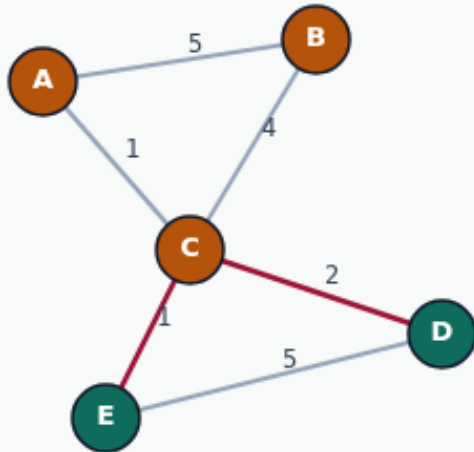
Final ABC|DE, cutsize 3

Pass 1 confirms local optimum

FIDUCCIA-MATTHEYSES BIPARTITION

Pass 1 confirms local optimum

Step 5 / 5 · pass1-stop · module02-03-fm-partition



Explanation

A second FM pass finds no improving move prefix. Stop. In real tools, FM often runs inside multilevel V-cycles with tighter balance and hyperedges.

- improved=false on pass 1
- Cell moves scale to large netlists
- Next: multilevel + hypergraph partition

```
pass 1: best_k=0  
stop
```

Pass 1 confirms local optimum

Browser lab track

- In the browser lab track, open the **fm-partition** lab from the tools shelf
- Load the starter graph, run the algorithm once
- Work the challenges that lock the goldens

Implement track

- In the implement track
- Parse the tiny graph, run the algorithm with a deterministic seed
- Match the browser goldens before you claim the checklist

Pitfalls

- Common traps
- For multilevel flows, verify coarsening before you blame the refiner

Your turn

- Complete the checklist for at least one track, preferably both
- Implement until your metrics match the starter goldens
- When you're ready, take the short quiz, then continue to the next module